A quenched QCD implementation using Fortran coarrays
Lab report - Projektpraktikum, M.Sc. Physik

Nicolas Lang

Abstract A fortran implementation of quenched QCD using coarrays is presented.

1 Introduction

2 QCD

2.1 The path integral quantization

2.2 Discretization on the lattice

2.3 The QCD action

3 The Metropolis Algorithm

4 Design choices

This section discusses the general design ideas behind the coarray-based quenched QCD implementation presented in this report.

4.1 Fortran coarrays

Coarrays provide a way to distribute data accross multiple program instances while retaining a concise syntax that abstracts the underlying inter-process or network communication. Formerly an extension of Fortran 95, coarrays are now part of the Fortran 2008 standard. Coarrays are variables with one or several *codimensions* and indices referencing data stored by another program instance, called *image*. *Images* represent copies of the program that run asynchronously and can possess further independent local variables. The coarray entries can be accessed from every image by specifying the respective *coindex* representing the image, where the data is stored. The size of the codimension is determined at runtime from the number of running images. The standard

Bergische Universität Wuppertal

provides for additional methods that enable the user to obtain the number of images and the index of the current image and to introduce a break point to synchronize images. The coarray support in the GNU Fortran compiler is based on the *OpenCoarrays* **opencoarrays** library which in turn is based on the Message Passing Interface (MPI).

4.2 Shortcomings of the current coarray implementation

The current implementation of coarrays in Fortran suffers from a number of shortcomings. Most notably, there is no notion of coarray pointers. This makes data encapsulation difficult. A new proposal called Coarray Fortran 2.0 **coarr2.0** addresses this and other issues, but it is not clear whether and when it will become part of the Fortran standard. Therefore, work-arounds need to be found.

4.3 Overview and general ideas

The goal of this

5 Implementation

This section is dedicated to the particular implementation of the aforementioned ideas. The derived types and operators are discussed first. Then a concise implementation of the metropolis algorithm is presented.

5.1 Types

A number of derived types is needed in order to hide underlying data structures from the user and to facilitate global field operations. These types can be grouped into field types, which keep a record of spatial and temporal indices, and field variable types, which are meaningful only with respect to a given field or in the context of reductions. The implementation makes use of object oriented features introduced in Fortran 2003.

Gauge field

The *gauge_field* type serves as an auxiliary handle to perform operations on the gauge field itself. It does not contain any data as a member but instead accesses the global module variable *data* from its type bound methods. The following code snippet shows the type definition:

Listing 1: Type definition of the *gauge_field* type

```
complex, allocatable :: data(:,:,:)[:]

type, public :: gauge_field
    integer, allocatable :: indices(:,:,:,:,:)
    logical :: tmp
    integer :: dir
    integer :: par
    integer :: x_low, x_up, y_low, y_up, z_low, z_up, t_low, t_up
    integer :: nx, ny, nz, nt
```

```
contains

    ! ...
    procedure :: initialize
    procedure :: update
    ! ...

end type gauge_field
```

Data is stored linearly. The *data* variable is an allocatable coarray of type complex with three dimensions and one codimension. It's first two dimensions are those of the complex matrices, running from 1 to 3 in the case of SU(3). The third dimension is the local index. The codimension (in square brackets) represents the index of the image. These indices will be refered to as *storage indices*. As field operators will often depend on link variables of the nearest neighbours, performing parallel operations requires lattice sites to be divided into sites of even or odd parity in these cases. Parity is defined by the euclidean indices as $p = (x+y+z+t)\ mod\ 2$. Thus, all sites of a given parity can be updated in parallel without affecting one another. It is then favourable from a performance perspective to align data of a given parity in memory in order to avoid cache misses. Data is therefore organized in the order $x, y, z, t, direction, parity$, i.e. link variables of a particular direction and parity that are next to each other in x are stored consecutively. Euclidean indices and link direction will be referred to as *field indices*. The translation between field indices and storage indices is done with the help of an index array. Its dimensions represent in this order: index/coindex, $x, y, z, t, direction$. Parity is not stored as an index since it can be calculated from the euclidean indices. The *gauge_field* type possesses two integer members *dir* and *par* which serve as flags on temporaries used in arithmetic operations. This is further explained when discussing operators. The integers x_{low}, x_{up} etc. define the boundaries of the current image, n_x, ..., n_t the lengths of the respective dimensions. The *initialize* method allocates the data array and initializes the matrices either to the unit matrix or random unitary 3×3 matrices. It also fills the index array and distributes the data to the available images. The latter is performed in the t and z dimensions. The *update* method updates all link variables of parity *pin* and direction *din* with values stored in the *su3_field_var* variable supplied as the third argument, if the corresponding link stored in the fourth argument of type *logical_field_var* is set to `.true.`.

Listing 2: Update subroutine of the *gauge_field* type. The subroutine *comp_dimensions* computes *d_low*, *d_up* and *step_size* from the values of the direction and parity flags

```
subroutine update(a, pin, din, fv, lfv, acc)
    class(gauge_field), intent(inout) :: a
    integer, intent(in) :: pin, din
    type(loc_su3_field_var), intent(inout) :: fv
    type(loc_logical_field_var), intent(in) :: lfv
    integer, intent(inout) :: acc
    integer :: i,j,k,l,d, idx
    integer :: d_low, d_up, da, la, step_size

    a%par = pin
    a%dir = din

    call check_dim_fv(a, size(fv%data)/9)
    call check_dim_fv(a, size(lfv%data))
    call comp_dimensions(a, fv, d_low, d_up, step_size)

    idx = 1
    do d = d_low, d_up                          ! directions
        do i = a%t_low, a%t_up                  ! t
            do j = a%z_low, a%z_up              ! z
                do k = a%y_low, a%y_up                                  ! y
                    do l = a%x_low-1+step_size, a%x_up, step_size       ! x
                        if (d == 0) then                ! specific direction
                            da = abs(a%dir)
                        else                            ! all directions
                            da = d
```

```
                        end if
                        if (step_size == 2) then    ! defined parity
                            la = l-abs(mod(i+j+k,2)-a%par)
                        else                        ! undefined parity
                            la = l
                        end if

                        if (lfv%data(idx)) then
                            if (fv%conj) then
                                data(:,:,a%indices(0,la,k,j,i,da)) &
                                [a%indices(1,la,k,j,i,da)] &
                                = conjg(transpose(fv%data(:,:,idx)))
                            else
                                data(:,:,a%indices(0,la,k,j,i,da)) &
                                [a%indices(1,la,k,j,i,da)] &
                                = fv%data(:,:,idx)
                            end if
                            acc = acc + 1
                        end if
                        idx = idx + 1

                    end do
                end do
            end do
        end do
    end do

    ! reset parity and direction again
    a%par = -1
    a%dir = 0

end subroutine
```

Field variables

Field variable types possess data members which are either arrays or coarrays of a given type and represent the results of operations on *gauge_fields* and other field variables. They do not have any notion of spatial and temporal indices or direction. This is intended since the operands of a binary operator creating the field variable could have different parities or directions. Therefore the indices of the result would not be well defined. Hence, field variables are only meaningful in operations with gauge fields or reductions, such as sums or mean values. Currently 3 types of field variables are implemented:

- SU(3)-matrix-valued: *su3_field_var*
- real-valued: *real_field_var*
- logical-valued: *logical_field_var*

Listing 3: Type definition of the *su3_field_var* type

```
type, public :: su3_field_var
    complex, allocatable :: data(:,:,:)[:]
    logical :: conj = .false.
contains
    procedure :: add
    procedure :: add_glob_loc
    procedure :: assign
    procedure :: increment
    procedure :: real_trace
    procedure :: init_rnd
```

```
    procedure :: init_zero
    generic :: operator (+) => add, add_glob_loc
    generic :: assignment (=) => assign
    generic :: operator (.rtrc.) => real_trace
end type su3_field_var
```

Additionally each field variable type comes in a local version, meaning that the data member is an arrray instead of a coarray. This is necessary since derived types containing coarrays cannot be function return values. An overloaded assignment operator on the (global) field variable types allows the assignment of local field variables to (global) field variables, i.e. the array is copied into the local part of the coarray on every image. Reductions are only possible on (global) field variables for obvious reasons. Increment and initialization routines are defined for the *su3_field_var* and *real_field_var* types. The . rtrc . operator calculates the real part of the trace on an *su3_field_var* and returns a *real_field_var*.

5.2 Operators

For convenient use of the *gauge_field* type and the field variable types in arithmetic and logical operations various operators are overloaded. On *gauge_field*s additionally select and shift operators are defined to specify which links should be used in subsequent operations.

Select and shift

As most operations are not intended to be applied to every link simultaneously select and shift operators are defined on *gauge_field*s. The select operator enables the user to specify a parity and a link direction. It is a binary operator where one of the operands is a *gauge_field* and the other an integer array with two entries, the first of which specifies parity and the second the link direction. If both parities are needed a value of -1 is used for the first entry. To select all directions the second entry is set to 0. Negative directions cause the respective links to be conjugated in the subsequent operation. The select operator creates a temporary and sets the parity and direction flags on it to the respective values.

Listing 4: Select operator of the *gauge_field* type

```
function select (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, 0, 0)
    tmp%par = b(1)
    tmp%dir = b(2)
    tmp%tmp = .true.
end function
```

The shift operator is used to select neighbouring sites (as needed by gauge invariant operators such as the plaquette). It is a binary operator whose operands are a *gauge_field* and an integer array with two entries. The first entry represents the shift direction, the second entry the shift distance. The shift operator returns a temporary copy of the current *gauge_field* where the index array has been shifted in the specified direction and by the specified distance.

Listing 5: Shift operator of the *gauge_field* type

```
function shift (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp
```

```
5
6       call a%copy_shift(tmp, b(1), b(2))
7       tmp%tmp = .true.
8   end function
```

Arithmetic and logical operators

Arithmetic and logical operators can be defined between two *gauge_field*s, between a *gauge_field* and an *su3_field_var* or between field variables. The abundance of types and possible combinations requires a large number of operator implementations, many of which only differ in the declaration part of the function. Since Fortran does not provide templates code duplication is unavoidable. Currently only the operators needed for the plaquette/staple are implemented. Since it is always possible to convert between local and global field variables, most of them are only available for the local versions of the field variables.

Listing 6: Multiplication operator between a *gauge_field* and an *su3_field_var*. The function multSu3 is a wrapper around the Basic Linear Algebra Subroutines (BLAS) function cgemm. The second and fourth argument indicate whether the matrix should be conjugated.

```
1  function multiply_fv(a, b) result (res)
2
3      ! ...
4
5      idx = 1
6      do d = d_low, d_up
7          do i = a%t_low, a%t_up
8              do j = a%z_low, a%z_up
9                  do k = a%y_low, a%y_up
10                     do l = a%x_low-1+step_size, a%x_up, step_size
11                         if (d == 0) then
12                             da = abs(a%dir)
13                         else
14                             da = d
15                         end if
16                         if (step_size == 2) then
17                             la = l-abs(mod(i+j+k,2)-a%par)
18                         else
19                             la = l
20                         end if
21
22                         res%data(:,:,idx) = multSu3( &
23                         data( :,:,a%indices(0,la,k,j,i,da)) &
24                         [a%indices(1,la,k,j,i,da)], &
25                         (a%dir < 0), &
26                         b%data(:,:,idx), &
27                         b%conj )
28                         idx = idx + 1
29
30                     end do
31                 end do
32             end do
33         end do
34     end do
35
36 end function
```

It is necessary that operands have the same field dimensions, i.e. if odd parity has been selected on one of the operands then parity (but not necessarily the same) needs to be set on the other. All operators act on all selected links simultaneously, iterating over them in the order they are stored in memory. Which operators are

currently implemented for which types can be seen in table 1. Logical operators produce *logical_field_var*s as

Table 1: Arithmetic and logical operators and possible operands

Operator	Types
add (+)	*gauge_field*, *su3_field_var*, *real_field_var*
subtract (-)	*gauge_field*, *su3_field_var*, *real_field_var*
multiply (*)	*gauge_field*, *su3_field_var*, scalars
less than (<)	*real_field_var*
less than or equal (<=)	*real_field_var*
equal (==)	*real_field_var*
greater than or equal (>=)	*real_field_var*
greater than (>)	*real_field_var*

their results. To facilitate operations including intrinsic types like scalars and to overload intrinsic functions like *exp* for field variables, an interface module is provided.

5.3 The metropolis module

The metropolis module provides a concise implementation of the metropolis algorithm using the types and operators described above. It contains the *staple* function and the *metro* subroutine as well as an array of 100 *su3_field_var*s containing update matrices close to unity. The *init_update_tab* subroutine allocates the update table. Matrices close to unity are produced from SU(2) generators following the method described in **gattringer** The staple function makes use of the *increment* method of *su3_field_var*s to avoid unnecessary temporaries.

Listing 7: Staple function

```
function staple(u, p, d)

    type(gauge_field), intent(in) :: u
    integer, intent(in) :: d, p
    integer :: i, s
    type(loc_su3_field_var) :: staple

    s = u%nx * u%ny * u%nz * u%nt / 2

    call staple%init_zero(s)
    do i = 1, 4
        if (i /= d) then
            call staple%increment( &
                (u.sel.[p,i]) &
                * (u.shift.[i,1].sel.[p,d]) &
                * (u.shift.[d,1].sel.[p,-i]) &
                + (u.shift.[i,-1].sel.[p,-i]) &
                * (u.shift.[i,-1].sel.[p,d]) &
                * (u.shift.[i,-1].shift.[d,1].sel.[p,i]) &
                )
        end if
    end do

end function staple
```

The metropolis routine initializes the update table based on the value of *eps* if it has not been initialized yet and then iterates over even and odd sites and all directions. In every iteration a candidate link *su3_field_variable* is produced from the gauge field and a randomly chosen field variable from the update table. The selected update field variable is conjugated in 50% of the cases to fulfill the detailed balance criterion. The action change

delta$_s$ is represented by a *real_field_var* and calculated from the old and new links making use of the real trace operator . rtrc .. Finally the *gauge_field*'s update routine is called using a randomly initialized *real_field_var* and the exponentiated action delta for the update condition, the former being required to be smaller or equal to the latter for an update to take place. The metropolis routine will also keep track of the acceptance rate.

Listing 8: Metro subroutine running one sweep for both parities and each direction

```
subroutine metro(u, eps, beta, acc)

    type(gauge_field), intent(inout) :: u
    real, intent(inout) :: acc
    real, intent(in) :: eps, beta
    type(loc_su3_field_var) :: stp, new_link
    type(loc_real_field_var) :: delta_s, rnd_field
    integer :: p, d, s, acc_count
    real :: rnd1, rnd2

    s = u%nx * u%ny * u%nz * u%nt / 2
    acc_count = 0

    ! generate link updates
    call init_update_tab(100, s, eps)

    ! iterate over both parities and all directions
    do p = 1,2
        do d = 1,4

        ! calc. staple
        stp = staple(u,p,d)
        stp%conj = .true.

        ! generate new link
        call random_number(rnd1)
        call random_number(rnd2)
        if (rnd1 >= 0.5) then
            update_tab(ceiling(rnd2*100))%conj = .true.
        else
            update_tab(ceiling(rnd2*100))%conj = .false.
        end if

        new_link = (u.sel.[p,d]) * update_tab(ceiling(rnd2*100))

        ! calculate action change
        delta_s = -beta / 3.0 * .rtrc.((new_link - (u.sel.[p,d])) * stp)

        ! accept with some prob. dep. on action
        call rnd_field%init_rnd(s)
        call u%update(p, d, new_link, (rnd_field <= exp(-delta_s)), acc_count)
        sync all

        end do
    end do

    acc = real(acc_count) / real((s*8))

end subroutine metro
```

6 Sample measurement

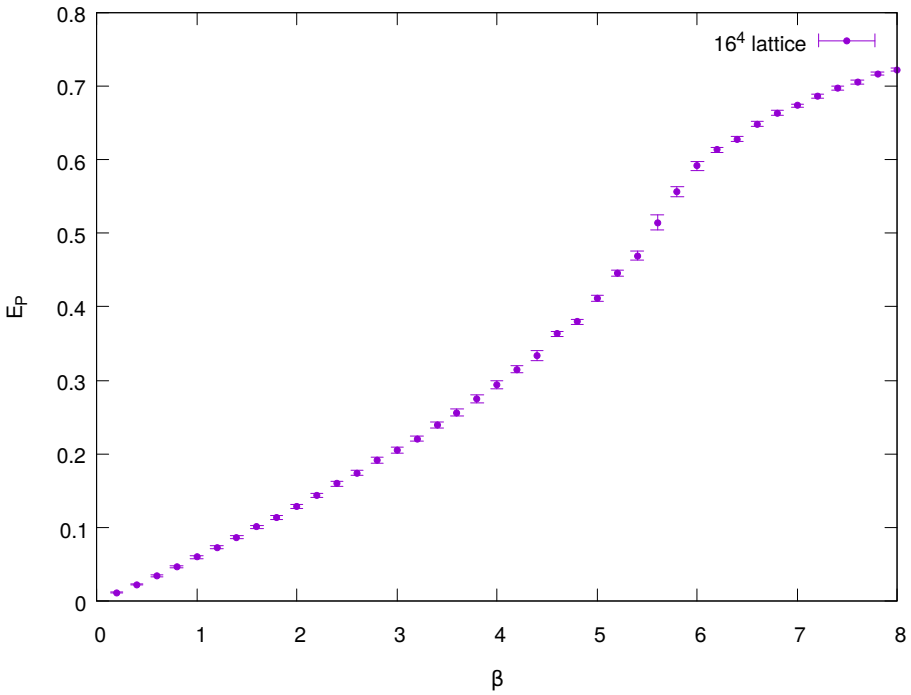


Fig. 1: β-Scan for a 16^4 lattice